```
import pandas as pd

# pd.DataFrame()
# pd.DataFrame(data, index=None, columns=None)
# data -> dictionary, list, array, DataFra,e
# index -> Row labels
# columns -> Column label

# 2 D table structure

df=pd.DataFrame({"pass":["yes","no","yes","yes","no"],
                 "revise": ["no","yes","no","yes","no"]})

print(" Full DataFrame")
print(df)
```

```
 Full DataFrame
  pass revise
0  yes     no
1   no    yes
2  yes     no
3  yes    yes
4   no     no
```

```
# filtering
# df[condition] -> boolean expression -> True / False

filtered = df [df["pass"]=="yes"]
print("only passed students")
print(filtered)
```

```
only passed students
  pass revise
0  yes     no
2  yes     no
3  yes    yes
```

```
print("Total Students", len(df))
print("Only passed Students", len(filtered))
```

```
Total Students 5
Only passed Students 3
```

```
# P(pass)

# p(pass) = no of passed students / total students

prob_pass= len(df[df["pass"]=="yes"]) / len(df)
print("P(Pass)", prob_pass)
```

```
P(Pass) 0.6
```

```
# series.value_counts()

# df["column"].value_counts(normalize=False, ascending=False)

print("COunt of pass column")
print(df["pass"].value_counts())

print("Prob of pass column")
print(df["pass"].value_counts(normalize=True))
```

```
COunt of pass column
pass
yes    3
no     2
Name: count, dtype: int64
Prob of pass column
pass
yes    0.6
no     0.4
Name: proportion, dtype: float64
```

```
# df -> revise, pass
# p(pass | revise)
```

```
import pandas as pd

df=pd.DataFrame({"pass":["yes","no","yes","yes","no"],
                 "revise": ["no","yes","no","yes","no"]})

print(" Full DataFrame")
print(df)

# step 1: filter the students who revised
revised_students=df[df["revise"]=="yes"]
print("Revised students")
print(revised_students)

# step 2: among the students who revised count how many have passed
passed_and_revised=revised_students[revised_students["pass"]=="yes"]
print("Pass and revise")
print(passed_and_revised)

# step 3: conditional prob
prob_pass_given_revised=len(passed_and_revised) / len(revised_students)
print(prob_pass_given_revised)
```

```
 Full DataFrame
  pass revise
0  yes     no
1   no    yes
2  yes     no
3  yes    yes
4   no     no
Revised students
  pass revise
1   no    yes
3  yes    yes
Pass and revise
  pass revise
3  yes    yes
0.5
```

```
prob_pass=(df["pass"]=="yes").mean()
print(prob_pass)
```

```
0.6
```

```
# pd.Series(data, index=None)
series=pd.Series(["yes","no","yes","no"])
print(series)

cus_series=pd.Series(["yes","no","yes","no"],index=["a","b","c","d"])
print(cus_series)
```

```
0    yes
1     no
2    yes
3     no
dtype: object
a    yes
b     no
c    yes
d     no
dtype: object
```

```
# monte carlo simulation -> simulate 1000 coin tosses

import random

heads=0
tails=0

for i in range(1000):
  # uniform dict -> 0 to 1
  if random.random() < 0.5:
    heads+=1
  else:
    tails+=1

# otherwise tails
```

```
# estimated prob
print("P(Head)", heads/1000)
print("P(tails)", tails/1000)
```

```
P(Head) 0.478
P(tails) 0.522
```

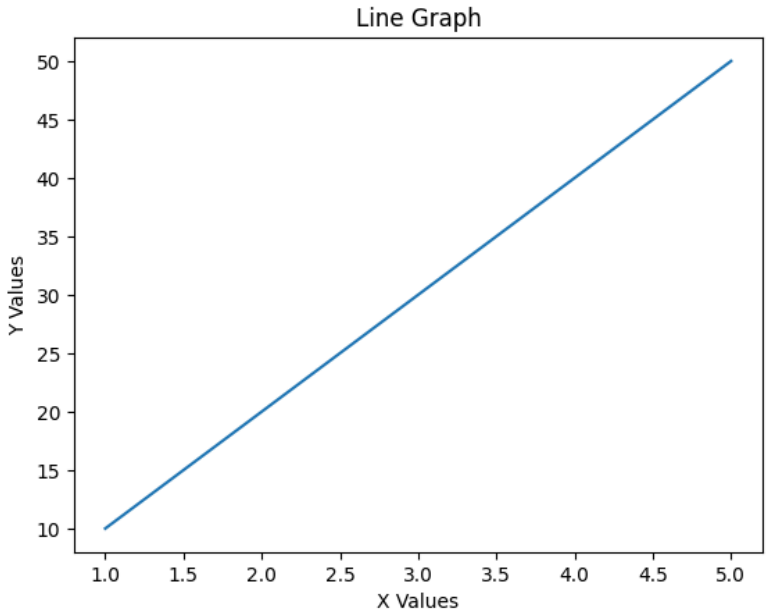
```
# monte carlo simulation -> simulate 1000 coin tosses

import random

heads=0
tails=0

for i in range(1000):
  # uniform dict -> 0 to 1
  if random.random() < 0.6:
    heads+=1
  else:
    tails+=1

# otherwise tails

# estimated prob
print("P(Head)", heads/1000)
print("P(tails)", tails/1000)
```

```
P(Head) 0.634
P(tails) 0.366
```

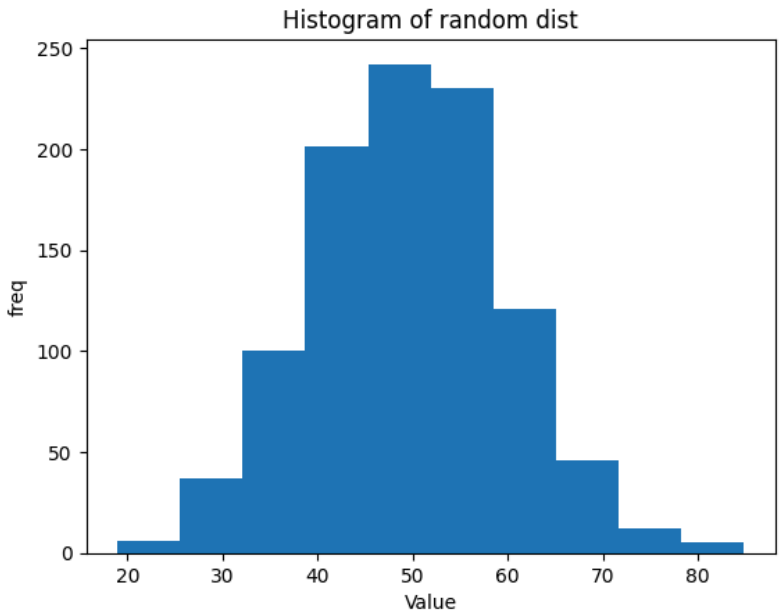
```
results=random.choices(["Head","Tail"], weights=[0.6,0.4],k=10)
print(results)
```

```
['Head', 'Tail', 'Head', 'Head', 'Tail', 'Head', 'Tail', 'Head', 'Tail', 'Head']
```

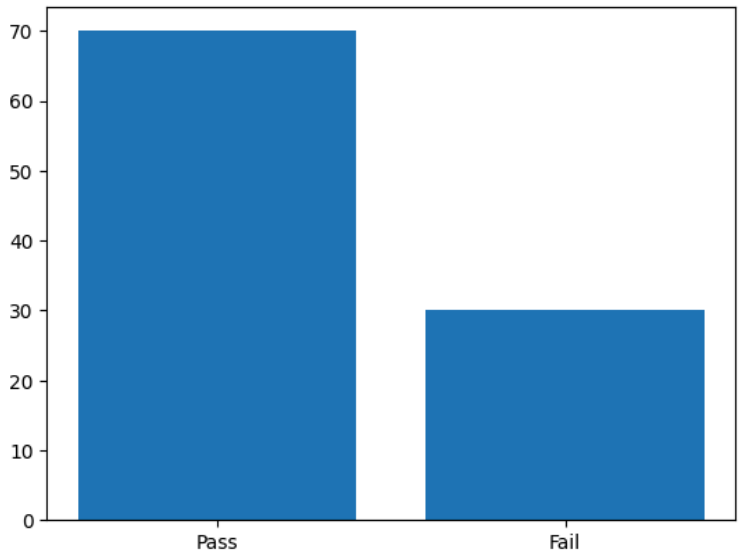
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# plt.plot()

# ply.plot(x,y) -> return line grph

x=[1,2,3,4,5]
y=[10,20,30,40,50]

plt.plot(x,y)
plt.title("Line Graph")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```



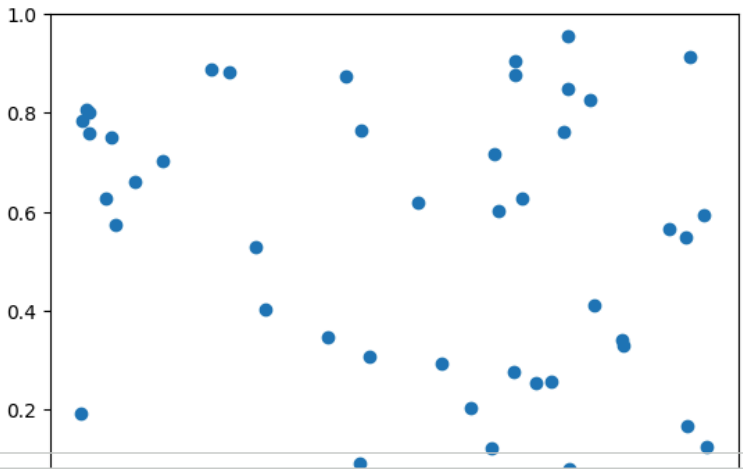
```
#plt.hist(data, bins=None) -> histogram

data=np.random.normal(50,10,1000)

plt.hist(data, bins=10)
plt.title("Histogram of random dist")
plt.xlabel("Value")
plt.ylabel("freq")
plt.show()
```



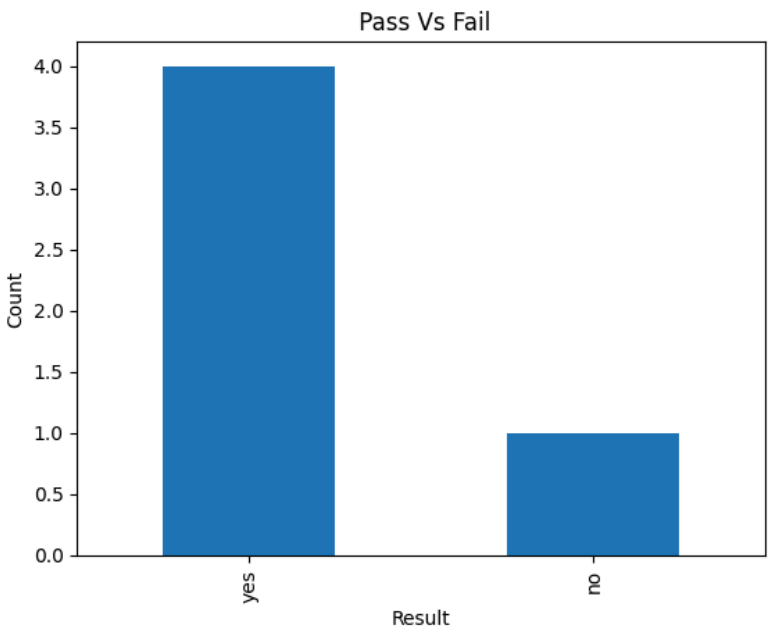

```
# bar

categories=["Pass","Fail"]
values=[70,30]

plt.bar(categories, values)
plt.show()
```




```
# plt.scatter()

x=np.random.rand(50)
y=np.random.rand(50)

plt.scatter(x,y)
plt.show()
```




```
# to plot form a dataframe

# df.plot(kind="bar")
df=pd.DataFrame({"pass":["yes","no","yes","yes","yes"]})

df["pass"].value_counts().plot(kind="bar")
plt.title(" Pass Vs Fail")
plt.xlabel("Result")
plt.ylabel("Count")
plt.show()
```



Statistics → Maths that deals
→ Collecting
→ Organize
→ Analyze
→ Interpret
→ Present
→ Informed decisions

Data

Marks
Height
Temp
Sales Number

Data

- Qualitative (Categorical)
  - Labels
  - Gender
  - Color
  - Dept
  - Pass | Fail
- Quantitative (Numer)
  - Discrete
    - → No. of students
  - Continuous
    - Height
    - Weight
    - Temperature

→ Summarize data
→ Mean
→ Median
→ Std dev

→ Decision Making

→ Data Science & ML

Linear Reg

Logistic Reg

Naive Bayes

Descriptive Stats - refers to methods

Org. sum, present

Main Components - ① Measures of Central Tendency

② Dispersion

③ Shape of Distribution

④ Percentiles

Inferential Stats — Use sample data

↓

Generalize

↓

Predictions (Population)

Population → Entire group of interest

Sample → Subset of population

Parameters – Numerical summary of population

App → Confidence Interval

Hypothesis Testing

Prob

Regression

Central Tendency Central or typical value of a dataset

X → Exact middle point
→ Exact average

→ What is the best value that represents dataset

50, 55, 60, 65, 70, 75, 80

⟹ Mean (Arithmetic Mean)

Average of all observations

$$\bar{x} = \frac{\sum x}{n}$$

$\bar{x}$ ← mean

$\sum x$ ← sum of values

n ← no. of observation

Properties

→ Every single data point

→ Overall balance point

→ Sensitive to outliers

50, 55, 60, 65, 70, 75, 80, 200

When → Data is symmetric

No extreme outliers

Median

Ascending order →

Middle → Median

if N is odd → middle value

N is even → avg of 2 middle values

Properties → Positional Center

50% value below Median

50% values above

Not affected by extreme Values

→ Income data
→ Real estate prices

→ Skewed data

Mode Value the occurs most frequently

Categorical data
Survey responses
Product pref

Measures of Dispersion How spread out, scattered, variable the data amount are around the central value

Dataset A : 50, 55, 60, 65, 70 Mean = 60

Dataset B : 10, 30, 60, 90, 110 Mean = 60

Range - Diff b/w max & min

→ Total width of data

Property - Very sensitive to outliers

Variance Variance $= \frac{\Sigma (x - \bar{x})^2}{n}$

Large → more spread

Small → less spread

Standard Deviation, Percentile → IQR